

ppttbarPaperPlots

May 26, 2026

This notebook creates the plots which are found in the paper: <https://arxiv.org/abs/2101.08944>. It also performs the statistical tests which are included in the supplementary material.

1 Load Required Libraries

```
[1]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import matplotlib.cm as cm
from scipy import stats

# Add utilityFunctions/ to easily use utility .py files
import sys
sys.path.append("../utilityFunctions")
```

2 Meta Parameters

```
[2]: save_figs = True # Global switch to turn on saving figures to files
N_slices = 1000 # Global number of slices to calculate SW distance with

data_directory = '../data/'
results_dataPath = '../experiments/ppttbar/otus_results-dataset=ppttbar_test.
↳npz'
dataset_name = 'ppttbar'
img_dir = dataset_name+'Images/'

np.random.seed(4) # Set numpy random seed
```

3 Get Truth Datasets and Model Results

The data was split into training, validation, and two testing sets. We load only the testing datasets here. We also load in the model results calculated on one of the testing datasets.

Note that we have two separate testing sets to ensure the independence of samples when performing statistical tests.

Specifically, we have 4 initial datasets: x_test1 , z_test1 , x_test2 , and z_test2 .

Test set 1 has been used to obtain network results. These results will be compared to the independent test set 2.

For testing the encoder's performance we will use the following convention: - $z_pred_te1 = E(x_test1, \epsilon)$

In plots this will be denoted as $x \rightarrow \tilde{z}$.

For testing the decoder's performance we will use the following convention: - $x_pred_truth_te1 = D(z_test1, \epsilon)$

In plots this will be denoted as $z \rightarrow \tilde{x}'$.

For testing the overall performance of the full encoder-decoder chain we will use the following convention: - $x_pred_te1 = D(E(x_test1, \epsilon), \epsilon)$

In plots this will be denoted as $x \rightarrow \tilde{z} \rightarrow \tilde{x}$.

Note that ϵ denotes the random noise input of the networks. Note that all ϵ are different random samples. This happens internally and thus won't be shown in what follows.

3.1 Load in Truth Data (Testing Dataset)

```
[3]: from func_utils import get_dataset, standardize

    ## Load Testing Truth Data##
    truthDataset = get_dataset(dataset_name+'_test', data_dir=data_directory)
    zData, xData = truthDataset['z_data'], truthDataset['x_data']

    ## Split data into the two testing sets ##
    f_test1 = int(0.5*zData.shape[0])

    x_test1 = xData[0:f_test1]
    z_test1 = zData[0:f_test1]

    x_test2 = xData[f_test1:]
    z_test2 = zData[f_test1:]

    ## Make sure testing sets have the same number of events ##
    assert(x_test1.shape[0] == x_test2.shape[0])
    assert(z_test1.shape[0] == z_test2.shape[0])
    print('z_test1 Shape, x_test1 Shape', z_test1.shape, x_test1.shape)
    print('z_test2 Shape, x_test2 Shape', z_test2.shape, x_test2.shape)
```

```
z_test1 Shape, x_test1 Shape (80000, 24) (80000, 24)
```

```
z_test2 Shape, x_test2 Shape (80000, 24) (80000, 24)
```

3.2 Get Results (Calculated on Testing Dataset)

```
[4]: ## Load Testing Results Data##
resultsDataset = np.load(results_dataPath)

# Load in all test data, with no selection cuts applied
zPred_raw      = resultsDataset['x_encoded']      # E(x)
xPred_raw      = resultsDataset['x_reconstructed'] # D(E(x))
xPredTruth_raw = resultsDataset['z_decoded']      # D(z)

## Get predicted z for x_test1, E(x_test1) ##
z_pred_te1 = zPred_raw[0:f_test1]

## Get predicted x for full autoencoder chain, D(E(x_test1)) ##
x_pred_te1 = xPred_raw[0:f_test1]

## Get predicted x for true z_test1, D(z_test1) ##
x_pred_truth_te1 = xPredTruth_raw[0:f_test1]

## Apply PT selection cuts ##
# These cuts require PT < 20 GeV
# These are similarly applied "by hand" in Delphes, so we do the same here. See
→ the paper for further discussion.
xPredMask      = resultsDataset['x_reconstructed_good_mask'] # Mask to select
→ xPred passing selection cuts
xPredTruthMask = resultsDataset['z_decoded_good_mask']      # Mask to select
→ xPredTruth passing selection cuts

# Two masks for test set 1 only
xPred_test1_mask      = xPredMask[0:f_test1]
xPredTruth_test1_mask = xPredTruthMask[0:f_test1]

# Data for examining xPred results
x_pred_te1      = x_pred_te1[xPred_test1_mask]
z_pred_te1_mskxPred = z_pred_te1[xPred_test1_mask]

# Data for examining xPredTruth results
x_pred_truth_te1      = x_pred_truth_te1[xPredTruth_test1_mask]
z_test1_mskxPredTruth = z_test1[xPredTruth_test1_mask]
x_test1_mskxPredTruth = x_test1[xPredTruth_test1_mask]

## Truncate data such that it is all the same size ##
```

```

n = np.min([x_pred_te1.shape[0], x_pred_truth_te1.shape[0]])

# Data for xPred results
x_pred_te1_trunc = x_pred_te1[0:n]
z_pred_te1_mskxPred_trunc = z_pred_te1_mskxPred[0:n]

# Data for xPredTruth results
x_pred_truth_te1_trunc = x_pred_truth_te1[0:n]
z_test1_mskxPredTruth_trunc = z_test1_mskxPredTruth[0:n]
x_test1_mskxPredTruth_trunc = x_test1_mskxPredTruth[0:n]

# True data that will be used for comparison with xPred and xPredTruth
x_test2_trunc = x_test2[0:n]

#-- For Cleaner Notation Below, we form these into compact datasets --#

# Data for making Encoder comparison plots (no cuts included)
# z_test2 and z_pred_te1 -> z and \tilde{z} in plots
EPlotData = [z_test2, z_pred_te1]

# Data for making Decoder comparison plots
# x_test2_trunc and x_pred_te1_trunc and x_predTruth_te1_trunc -> x and
# \tilde{x} and \tilde{x}' in plots
DPlotData = [x_test2_trunc, x_pred_te1_trunc, x_pred_truth_te1_trunc]

# Data for making Transport plots (pairs are important here)
# z_pred_te1_mskxPred_trunc and x_pred_te1_trunc -> \tilde{z}
# \rightarrow \tilde{x}
# z_test1_mskxPredTruth_trunc and x_test1_mskxPredTruth_trunc -> z \rightarrow
# \tilde{x}
# z_test1_mskxPredTruth_trunc and x_pred_truth_te1_trunc -> z \rightarrow
# \tilde{x}
TransportPlotData = [[z_pred_te1_mskxPred_trunc, x_pred_te1_trunc],
                     [z_test1_mskxPredTruth_trunc, x_test1_mskxPredTruth_trunc],
                     [z_test1_mskxPredTruth_trunc, x_pred_truth_te1_trunc]]

```

```

[5]: print(r"$z$ and $\tilde{z}$: ", EPlotData[0].shape,
      ↪EPlotData[1].shape)
print(r"$x$ and $\tilde{x}$ and $\tilde{x}^\prime$: ", DPlotData[0].shape,
      ↪DPlotData[1].shape, DPlotData[2].shape)
print("")
print(r"$\tilde{z}$ -> $\tilde{x}$: ", TransportPlotData[0][0].
      ↪shape, TransportPlotData[0][1].shape)
print(r"$z$ -> $x$: ", TransportPlotData[1][0].
      ↪shape, TransportPlotData[1][1].shape)

```

```
print(r"$z$ -> $\tilde{x}$: ", TransportPlotData[2][0].
      ↪shape, TransportPlotData[2][1].shape)
```

```
$z$ and $\tilde{z}$: (80000, 24) (80000, 24)
$x$ and $\tilde{x}$ and $\tilde{x}^\prime$: (47856, 24) (47856, 24) (47856, 24)

$\tilde{z}$ -> $\tilde{x}$: (47856, 24) (47856, 24)
$z$ -> $x$: (47856, 24) (47856, 24)
$z$ -> $\tilde{x}$: (47856, 24) (47856, 24)
```

4 Define Statistical Tests That We Will Use

For each one-dimensional distribution we will perform the following statistical tests:

- χ^2
- 2-sample, 2-sided Kolmogorov-Smirnov test

We will also report the 1D Wasserstein distance.

z-space plots will compare - z_test2 with z_pred_tel

In plots this will be denoted as z vs $\tilde{z}$.

x-space plots will compare - x_test2 with x_pred_tel

In plots this will be denoted as x vs $\tilde{x}$. - x_test2 with x_pred_truth_tel

In plots this will be denoted as x vs $\tilde{x}'$.

4.1 Import statistical test functions

```
[6]: from func_utils import runStatAnalysis
      from func_utils import sliced_wd_numpy as avSWDistance
```

4.2 Calculate SW Distance results (global metric) over N_Slices

4.2.1 z-space results

```
[7]: print(r"SW distance $z$ vs $\tilde{z}$: ", avSWDistance(EPlotData[0],
      ↪EPlotData[1], N_slices))
```

```
SW distance $z$ vs $\tilde{z}$: 22.3497831012193
```

4.2.2 x-space results

```
[8]: print(r"SW distance $x$ vs $\tilde{x}$: ", avSWDistance(DPlotData[0],
      ↪DPlotData[1], N_slices))
      print(r"SW distance $x$ vs $\tilde{x}^\prime$: ", avSWDistance(DPlotData[0],
      ↪DPlotData[2], N_slices))
```

```
SW distance $x$ vs $\tilde{x}$: 231.7597791579941
```

```
SW distance $x$ vs $\tilde{x}^\prime$: 120.07172680116433
```

4.3 Run statistical analysis for Principal Axis data

We run the statistical analysis on the principal axis plots that are shown in the paper. These plots are for the b quark's p_Y , p_Z , and E in z-space and the leading jet's p_Y , p_Z , and E in x-space.

We first find the correct binning such that no bin has less than 5 entries (necessary for χ^2 test). We store this binning in a list to use in later plotting.

4.3.1 z-space Find Correct Binning

```
[9]: # Want to find binsY such that n has no bins with <5 entries
zbinsY = [-250+10*i for i in range(50+1)]
n, _, _ = plt.hist(EPlotData[1][:,9], bins=zbinsY, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 12.  16.  22.  23.  20.  38.  39.  80.
 80. 128. 163. 252.
 369. 560. 769. 1211. 1695. 2381. 2897. 3673. 4565. 4986. 5218. 5333.
 5388. 5405. 5369. 5197. 4711. 4411. 3915. 3112. 2387. 1720. 1211. 797.
 532. 349. 241. 165. 107. 94. 72. 51. 32. 22. 25. 13.
 15. 7.]
```

Do any bins have less than 5 entries? False

```
[10]: # Want to find binsZ such that n has no bins with <5 entries
zbinsZ = [-700+25*i for i in range(56+1)]
n, _, _ = plt.hist(EPlotData[1][:,10], bins=zbinsZ, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 79.  95. 110. 131. 144. 164. 178. 188.
212. 245. 265. 295.
 389. 467. 510. 613. 658. 786. 1000. 1149. 1451. 1790. 2335. 2951.
 3913. 5212. 6571. 7402. 7428. 6331. 5091. 3867. 2917. 2183. 1672. 1399.
 1130. 943. 813. 695. 592. 509. 477. 417. 354. 285. 263. 221.
 217. 181. 174. 138. 139. 132. 95. 87.]
```

Do any bins have less than 5 entries? False

```
[11]: # Want to find binsE such that n has no bins with <5 entries
zbinsE = [0 + 25*i for i in range(32+1)]
n, _, _ = plt.hist(EPlotData[1][:,11], bins=zbinsE, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
```

```
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 677.  5772. 12668. 14017. 10590.  7393.
 5177.  3793.  2963.  2343.
 1904.  1589.  1375.  1125.  1021.   893.   698.   618.   565.   466.
  426.   361.   347.   295.   288.   257.   199.   184.   167.   160.
  139.   142.]
Do any bins have less than 5 entries? False
```

4.3.2 x-space Find Correct Binning

```
[12]: xbinsY = zbinsY
n, _, _ = plt.hist(DPlotData[1][:,9], bins=xbinsY, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 37.  49.  65.  56.  85. 104. 127. 186.
 216. 271. 368. 493.
 646. 908. 1128. 1529. 1782. 2018. 2343. 2319. 2067. 1901. 1762. 1644.
 1702. 1632. 1699. 1792. 1906. 2157. 2385. 2207. 2127. 1761. 1360. 1086.
  843.  664.  513.  363.  270.  237.  157.  144.  105.   88.   56.   43.
   37.   29.]
Do any bins have less than 5 entries? False
```

```
[13]: xbinsZ = zbinsZ
n, _, _ = plt.hist(DPlotData[1][:,10], bins=xbinsZ, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 82.  83. 106.  97. 118. 141. 171. 161.
 175. 219. 252. 270.
 334. 361. 409. 478. 568. 640. 792. 864. 1003. 1282. 1494. 1781.
 2226. 2691. 3113. 3375. 3323. 2960. 2514. 2147. 1733. 1480. 1158.  998.
  831.  726.  614.  533.  457.  404.  341.  289.  292.  274.  236.  187.
  206.  163.  160.  128.  103.  107.   78.   84.]
Do any bins have less than 5 entries? False
```

```
[14]: xbinsE = [0] + [50 + 25*i for i in range(30+1)]
n, _, _ = plt.hist(DPlotData[1][:,11], bins=xbinsE, histtype='step',
    ↪density=False)
```

```

print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot

```

```

Number of entries in each bin: [ 200. 2827. 6175. 6622. 5508. 4426. 3440. 2740.
 2126. 1798. 1435. 1235.
 1039.  895.  736.  674.  608.  501.  462.  395.  376.  346.  283.  227.
  217.  194.  171.  172.  150.  162.  130.]
Do any bins have less than 5 entries? False

```

4.3.3 Run statistical analysis for z-space principal axis data

```

[15]: ## Define suitable binning for PA data (obtained from above) ##
bins_py = zbinsY
bins_pz = zbinsZ
bins_E  = zbinsE

zbinsList = [bins_py, bins_pz, bins_E]

[16]: ## z-space PA data ##
zStatsList = []

for j in range(len(zbinsList)):

    k = (j+1) + 4*2

    zStatsList.append(runStatAnalysis(EPlotData[1][:,k], EPlotData[0][:,k],
↪zbinsList[j]))

[17]: ## Demonstration of how to access statistical test information ##
print(len(zStatsList))
print('W1D, KSstat, KSpval, RC2, dof, chi2pval')
print(zStatsList[0])
print("")
print('W1D: %.2E'% zStatsList[0][0])
print('(KSstat, pval): (%.2E, %.3f)' % (zStatsList[0][1], zStatsList[0][2]))
print('(RC2, dof, pval): (%.3f, %d, %.3f)' % (zStatsList[0][3],
↪zStatsList[0][4], zStatsList[0][5]))

```

```

3
W1D, KSstat, KSpval, RC2, dof, chi2pval
(15.832263767225289, 0.012500000000000067, 7.453306344156349e-06,
7.417748350780833, 49, 0.0)

W1D: 1.58E+01
(KSstat, pval): (1.25E-02, 0.000)
(RC2, dof, pval): (7.418, 49, 0.000)

```


4.3.4 Run statistical analysis for x-space principal axis data

```
[18]: bins_py = xbinsY
      bins_pz = xbinsZ
      bins_E = xbinsE

      xbinsList = [bins_py, bins_pz, bins_E]

[19]: ## x-space PA data ##
      xStatsList = []
      xStatsListTruth = []

      for j in range(len(xbinsList)):

          k = (j+1) + 4*2

          xStatsList.append(runStatAnalysis(DPlotData[1][:,k], DPlotData[0][:,k],
      ↪xbinsList[j]))

          xStatsListTruth.append(runStatAnalysis(DPlotData[2][:,k], DPlotData[0][:
      ↪,k], xbinsList[j]))

[20]: ## Demonstration of how to access statistical test information ##
      print(len(xStatsList))
      print('W1D, KSstat, KSpval, RC2, dof, chi2pval')
      print(xStatsList[0])
      print("")
      print('W1D: %.2E' % xStatsList[0][0])
      print('(KSstat, pval): (%.2E, %.3f)' % (xStatsList[0][1], xStatsList[0][2]))
      print('(RC2, dof, pval): (%.3f, %d, %.3f)' % (xStatsList[0][3],
      ↪xStatsList[0][4], xStatsList[0][5]))
```

```
3
W1D, KSstat, KSpval, RC2, dof, chi2pval
(23.971607333016532, 0.01659144098963561, 3.8001996041226287e-06,
2.3948165978810323, 49, 1.553509400009645e-07)
```

```
W1D: 2.40E+01
(KSstat, pval): (1.66E-02, 0.000)
(RC2, dof, pval): (2.395, 49, 0.000)
```

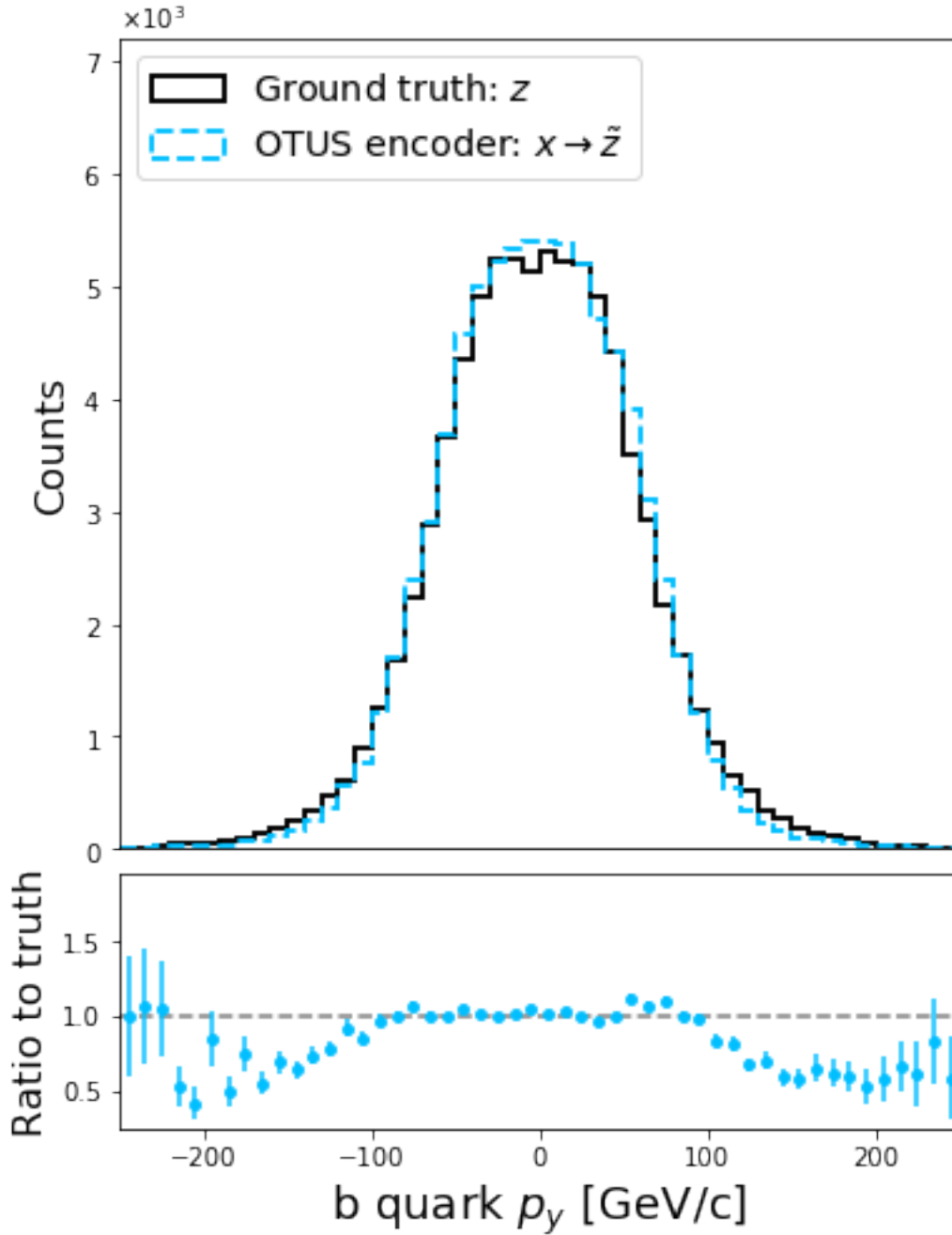
5 Make Principal Axes Plots

5.1 Load Plotting Function

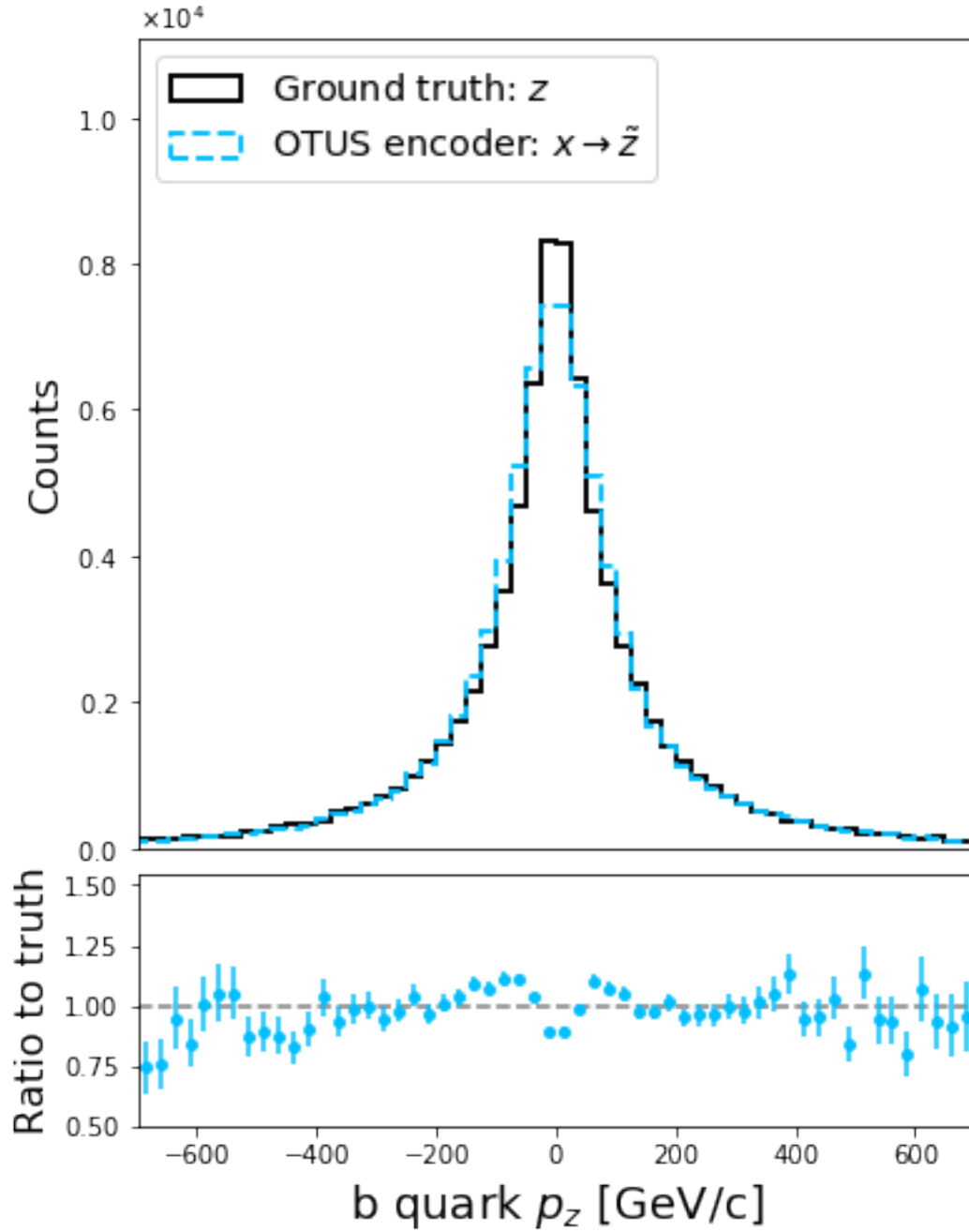
```
[21]: from plot_utils import ratioPlotSingle, ratioPlotDouble, transportPlot
```

5.2 Create Principal Axes Plots z-space

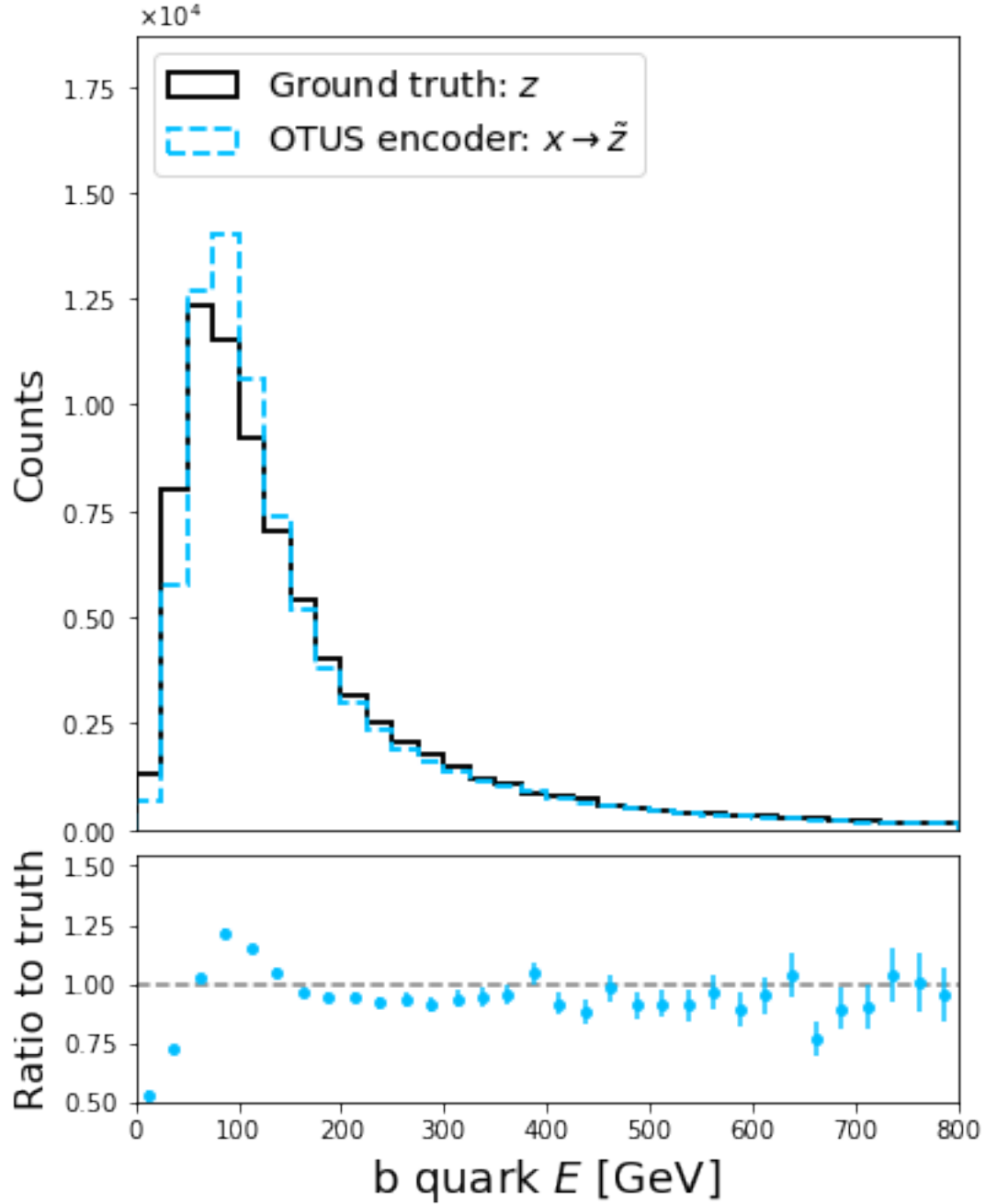
```
[22]: ## Plot this for the b Quark's py ##
ratioPlotSingle(EPlotData[0][:,9], EPlotData[1][:,9], zbinsList[0], (-250,250),
               ↪(0,7000), (0.25, 1.95),
               ↪r'b quark $p_y$ [GeV/c]', legend=True, save_figs=save_figs,
               ↪img_dir=img_dir) # zStatsList[0],
```



```
[23]: ## Plot this for the b Quark's pz ##
ratioPlotSingle(EPlotData[0][:,10], EPlotData[1][:,10], zbinsList[1],
               (-700,700), (0,7000), (0.5, 1.55),
               r'b quark $p_z$ [GeV/c]', legend=True, save_figs=save_figs,
               img_dir=img_dir) #zStatsList[1],
```

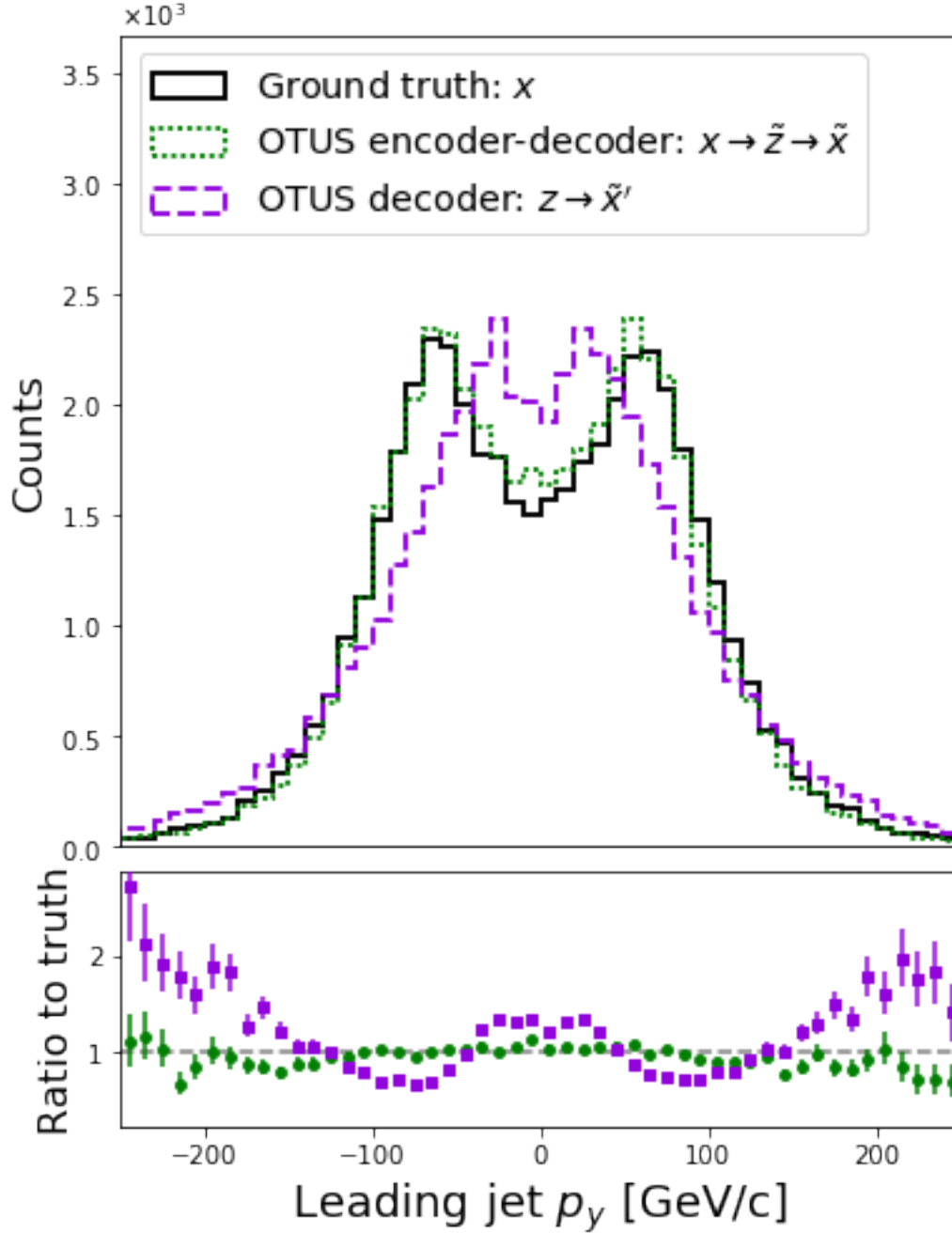


```
[24]: ## Plot this for the b Quark's E ##
ratioPlotSingle(EPlotData[0][:,11], EPlotData[1][:,11], zbinsList[2], (0,800),
↳(0,7000), (.5,1.55),
↳r'b quark $E$ [GeV]', legend=True, save_figs=save_figs,
↳img_dir=img_dir) # zStatsList[2],
```

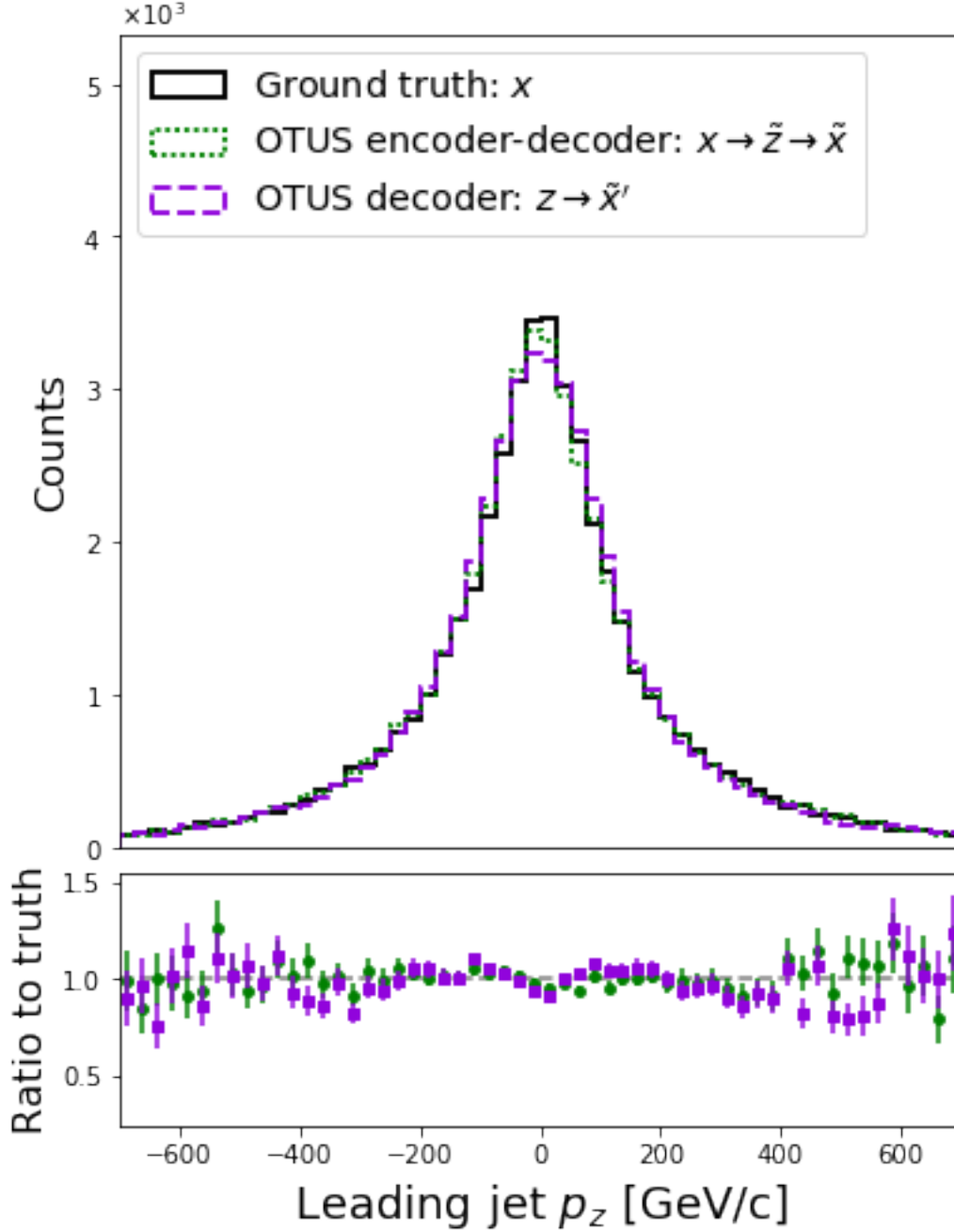


5.3 Principal Axes Plots x space

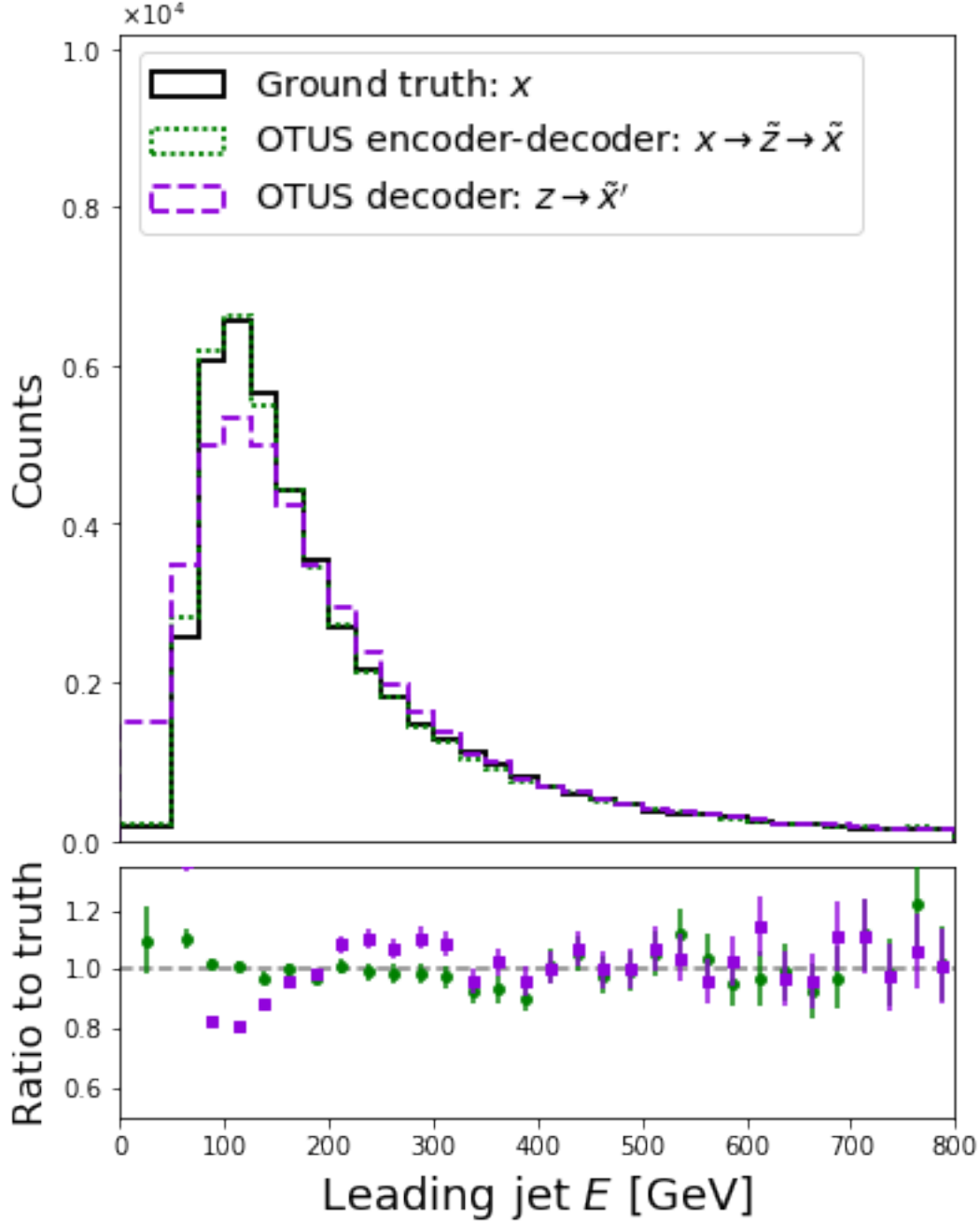
```
[25]: ## Plot this for the positron's py ##
ratioPlotDouble(DPlotData[0][:,9], DPlotData[1][:,9], DPlotData[2][:,9],
↳xbinsList[0],
               (-250,250), (0,7000), (0.25,2.85),
               r'Leading jet $p_y$ [GeV/c]', legend=True,
↳save_figs=save_figs, img_dir=img_dir) #xStatsList[0], xStatsListTruth[0],
```



```
[26]: ## Plot this for the positron's pz ##
ratioPlotDouble(DPlotData[0][:,10], DPlotData[1][:,10], DPlotData[2][:,10],
↳xbinsList[1],
          (-700,700), (0,7000), (0.25,1.55),
          r'Leading jet $p_z$ [GeV/c]', legend=True,
↳save_figs=save_figs, img_dir=img_dir) #xStatsList[1], xStatsListTruth[1],
```



```
[27]: ## Plot this for the positron's E ##
ratioPlotDouble(DPlotData[0][:,11], DPlotData[1][:,11], DPlotData[2][:,11],
↳xbinsList[2],
(0,800), (0,7000), (0.5,1.35),
r'Leading jet $E$ [GeV]', legend=True, save_figs=save_figs,
↳img_dir=img_dir) #xStatsList[2], xStatsListTruth[2],
```



6 Calculate Derived Quantities

6.1 Calculate transverse momentum (p_T)

Transverse momentum, p_T , is a quantity derived from two principal axes: p_x and p_y . Namely, $p_T = p_x^2 + p_y^2$

```
[28]: def calcPT(px, py):  
       return np.sqrt(px**2 + py**2)  
  
[29]: ## Calculate PT for x-space data for leading jet ##  
xtest_jet1_PT = calcPT(DPlotData[0][:,8], DPlotData[0][:,9])  
xpred_jet1_PT = calcPT(DPlotData[1][:,8], DPlotData[1][:,9])  
xpredtruth_jet1_PT = calcPT(DPlotData[2][:,8], DPlotData[2][:,9])  
  
jet1_PTData = [xtest_jet1_PT, xpred_jet1_PT, xpredtruth_jet1_PT]  
  
assert(np.min(jet1_PTData[0])>20)  
assert(np.min(jet1_PTData[1])>20)  
assert(np.min(jet1_PTData[2])>20)
```

6.2 Calculate Invariant Mass Derived Quantities

In x-space, invariant mass checks are somewhat complicated as we don't know which jets correspond to which quarks. There are 5 in total.

- 1) t tbar pair mass
- 2) Leptonic W mass
- 3) Hadronic W mass
- 4) Leptonic t mass
- 5) Hadronic t mass

6.2.1 Import code to calculate these quantities

```
[30]: from top_masses import ttbar_masses  
  
# Note that ttbar_masses(x) where x is a 24-length vector (i.e. one sample from  
# x-space)
```

6.2.2 Calculate for x in DPlotData[0] data

```
[31]: # Loop over all events and calculate the masses in each case  
xtest_massesArr = np.array([ttbar_masses(DPlotData[0][i,:]) for i in  
# range(DPlotData[0].shape[0])])
```



```
xtest_IM_ttbar, xtest_IM_Wlep, xtest_IM_Whad, xtest_IM_tlep, xtest_IM_thad =
    ↪xtest_massesArr[:,0], xtest_massesArr[:,1], xtest_massesArr[:,2],
    ↪xtest_massesArr[:,3], xtest_massesArr[:,4]
```

6.2.3 Calculate for $\tilde{x}$ in DPlotData[1] data

```
[32]: # Loop over all events and calculate the masses in each case
xpred_massesArr = np.array([ttbar_masses(DPlotData[1][i,:]) for i in
    ↪range(DPlotData[1].shape[0])])

xpred_IM_ttbar, xpred_IM_Wlep, xpred_IM_Whad, xpred_IM_tlep, xpred_IM_thad =
    ↪xpred_massesArr[:,0], xpred_massesArr[:,1], xpred_massesArr[:,2],
    ↪xpred_massesArr[:,3], xpred_massesArr[:,4]
```

6.2.4 Calculate for $\tilde{x}'$ in DPlotData[2] data

```
[33]: # Loop over all events and calculate the masses in each case
xpredtruth_massesArr = np.array([ttbar_masses(DPlotData[2][i,:]) for i in
    ↪range(DPlotData[2].shape[0])])

xpredtruth_IM_ttbar, xpredtruth_IM_Wlep, xpredtruth_IM_Whad,
    ↪xpredtruth_IM_tlep, xpredtruth_IM_thad = xpredtruth_massesArr[:,0],
    ↪xpredtruth_massesArr[:,1], xpredtruth_massesArr[:,2], xpredtruth_massesArr[
    ↪:,3], xpredtruth_massesArr[:,4]
```

6.2.5 Form into lists for compact notation

```
[34]: ttbarPlotData = [xtest_IM_ttbar, xpred_IM_ttbar, xpredtruth_IM_ttbar]
WlepPlotData = [xtest_IM_Wlep, xpred_IM_Wlep, xpredtruth_IM_Wlep]
WhadPlotData = [xtest_IM_Whad, xpred_IM_Whad, xpredtruth_IM_Whad]
tlepPlotData = [xtest_IM_tlep, xpred_IM_tlep, xpredtruth_IM_tlep]
thadPlotData = [xtest_IM_thad, xpred_IM_thad, xpredtruth_IM_thad]
```

7 Find correct binning for Derived Quantities

7.1 Find correct binning for p_T

```
[35]: PTbins1 = [20] + [40 + 10*i for i in range(36)]

n, _, _ = plt.hist(xtest_jet1_PT, bins=PTbins1, histtype='step', density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))
n, _, _ = plt.hist(xpred_jet1_PT, bins=PTbins1, histtype='step', density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))
```

```
plt.close() # Don't show plot
```

Number of entries in each bin: [144. 873. 2275. 4224. 5394. 5640. 5254. 4608. 3864. 3007. 2487. 1948.

1550. 1287. 979. 793. 575. 465. 379. 320. 270. 177. 177. 141.

121. 101. 91. 86. 75. 65. 51. 58. 55. 31. 29. 31.]

Do any bins have less than 5 entries? False

Number of entries in each bin: [247. 1026. 2701. 4602. 5542. 5701. 5316. 4556. 3782. 2996. 2339. 1785.

1387. 1124. 874. 709. 564. 442. 324. 302. 221. 188. 149. 116.

120. 61. 72. 80. 65. 54. 48. 49. 47. 26. 31. 20.]

Do any bins have less than 5 entries? False

7.2 Find correct binning for invariant mass derived quantities

```
[36]: # Want to find bins such that n has no bins with <5 entries
bins_ttbar = [200]+[300 + 50*i for i in range(43+1)]
n, _, _ = plt.hist(ttbarPlotData[1], bins=bins_ttbar, histtype='step',
    density=False)

print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

Number of entries in each bin: [301. 2125. 5221. 6378. 6169. 5085. 4251. 3467. 2678. 2100. 1753. 1374.

1070. 875. 698. 651. 517. 412. 354. 329. 287. 235. 158. 158.

131. 142. 109. 86. 85. 70. 61. 59. 41. 43. 40. 35.

38. 23. 19. 18. 16. 29. 15. 11.]

Do any bins have less than 5 entries? False

```
[37]: # Want to find bins such that n has no bins with <5 entries
bins_Wlep = [80 + 0.4*i for i in range(4+1)] + [82 + 2*i for i in range(34+1)]
n, _, _ = plt.hist(WlepPlotData[1], bins=bins_Wlep, histtype='step',
    density=False)

print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot
```

Number of entries in each bin: [23147. 12352. 269. 264. 207. 1240. 1085. 1007. 943. 804.

713. 594. 531. 473. 422. 340. 329. 278. 261. 196.

187. 159. 155. 142. 134. 116. 89. 91. 70. 71.]

```

73.    56.    54.    51.    52.    51.    44.    31.    41.]
Do any bins have less than 5 entries? False

```

```

[38]: # Want to find bins such that n has no bins with <5 entries
#bins_Whad = [0 + 5*i for i in range(45+1)]
bins_Whad = [20 + 5*i for i in range(36+1)]
n, _, _ = plt.hist(WhadPlotData[1], bins=bins_Whad, histtype='step',
    ↪density=False)
#n, _, _ = plt.hist(WhadPlotData[0], bins=bins_Whad, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot

```

```

Number of entries in each bin: [ 46.   80.  128.  188.  311.  437.  738. 1198.
1904. 3114. 4672. 6406.
 6909. 5262. 3816. 2683. 2001. 1470. 1156.  952.  743.  634.  536.  422.
 343.  289.  254.  175.  151.  122.  77.  95.  73.  46.  46.  38.]
Do any bins have less than 5 entries? False

```

```

[39]: # Want to find bins such that n has no bins with <5 entries
bins_tlep = [80] + [100 + 10*i for i in range(30+1)]
#bins_tlep = [80, 90] + [100 + 5*i for i in range(54+1)] + [380, 390, 400]
n, _, _ = plt.hist(tlepPlotData[1], bins=bins_tlep, histtype='step',
    ↪density=False)
#n, _, _ = plt.hist(tlepPlotData[0], bins=bins_tlep, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

plt.close() # Don't show plot

```

```

Number of entries in each bin: [ 29.  113.  265.  712. 1589. 3200. 5699. 8179.
8365. 6314. 4092. 2432.
 1575. 1014.  783.  636.  534.  387.  271.  243.  197.  183.  141.  113.
  96.   86.   69.   61.   47.   51.   39.]
Do any bins have less than 5 entries? False

```

```

[40]: # Want to find bins such that n has no bins with <5 entries
bins_thad = [60 + 20*i for i in range(42+1)]
n, _, _ = plt.hist(thadPlotData[1], bins=bins_thad, histtype='step',
    ↪density=False)
n, _, _ = plt.hist(thadPlotData[0], bins=bins_thad, histtype='step',
    ↪density=False)
print("Number of entries in each bin: ",n)
print("Do any bins have less than 5 entries? ",np.any(n < 5))

```

```
plt.close() # Don't show plot
```

```
Number of entries in each bin: [ 45. 271. 1014. 2867. 6372. 9374. 6776. 4040.
2748. 2118. 1713. 1416.
1110. 922. 784. 654. 581. 530. 453. 424. 337. 302. 297. 222.
235. 200. 177. 134. 162. 132. 119. 102. 93. 126. 72. 69.
75. 66. 49. 44. 56. 47.]
Do any bins have less than 5 entries? False
```

8 Run Statistical Analysis for Derived Quantities

8.1 Run Statistical Analysis for p_T

```
[41]: xPTstatsList = []
      xPTstatsListTruth = []

      # Jet 1
      xPTstatsList.append(runStatAnalysis(jet1_PTData[1], jet1_PTData[0], PTbins1))
      xPTstatsListTruth.append(runStatAnalysis(jet1_PTData[2], jet1_PTData[0],
      ↪PTbins1))

      # Jet 4
      #xPTstatsList.append(runStatAnalysis(jet4_PTData[1], jet4_PTData[0], PTbins4))
      #xPTstatsListTruth.append(runStatAnalysis(jet4_PTData[2], jet4_PTData[0],
      ↪PTbins4))
```

8.2 Run Statistical Analysis for Invariant Mass Derived Quantities

```
[42]: xIMstatsList = []
      xIMstatsListTruth = []

      # ttbar
      xIMstatsList.append(runStatAnalysis(ttbarPlotData[1], ttbarPlotData[0],
      ↪bins_ttbar))
      xIMstatsListTruth.append(runStatAnalysis(ttbarPlotData[2], ttbarPlotData[0],
      ↪bins_ttbar))

      # Wlep
      xIMstatsList.append(runStatAnalysis(WlepPlotData[1], WlepPlotData[0],
      ↪bins_Wlep))
      xIMstatsListTruth.append(runStatAnalysis(WlepPlotData[2], WlepPlotData[0],
      ↪bins_Wlep))

      # Whad
      xIMstatsList.append(runStatAnalysis(WhadPlotData[1], WhadPlotData[0],
      ↪bins_Whad))
```

```

xIMstatsListTruth.append(runStatAnalysis(WhadPlotData[2], WhadPlotData[0],
↳bins_Whad))

# tlep
xIMstatsList.append(runStatAnalysis(tlepPlotData[1], tlepPlotData[0],
↳bins_tlep))
xIMstatsListTruth.append(runStatAnalysis(tlepPlotData[2], tlepPlotData[0],
↳bins_tlep))

# thad
xIMstatsList.append(runStatAnalysis(thadPlotData[1], thadPlotData[0],
↳bins_thad))
xIMstatsListTruth.append(runStatAnalysis(thadPlotData[2], thadPlotData[0],
↳bins_thad))

```

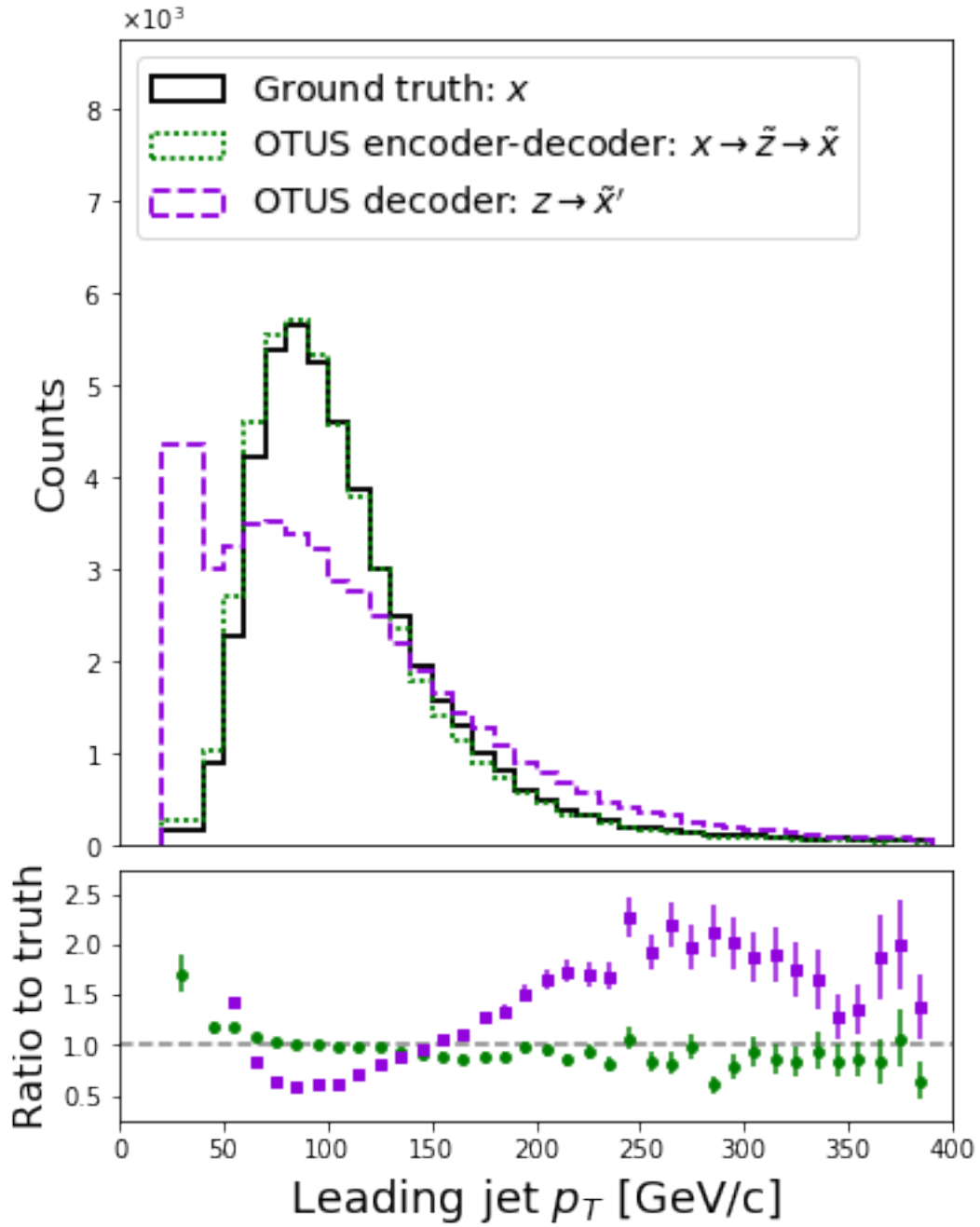
9 Make Plots for Derived Quantities

9.1 Make plots for p_T

```

[43]: ratioPlotDouble(jet1_PTData[0], jet1_PTData[1], jet1_PTData[2], PTbins1,
                    (0,400), (0,5000), (0.25,2.75),
                    r'Leading jet $p_T$ [GeV/c]', legend=True,
↳save_figs=save_figs, img_dir=img_dir) #xPTstatsList[0],
↳xPTstatsListTruth[0],

```



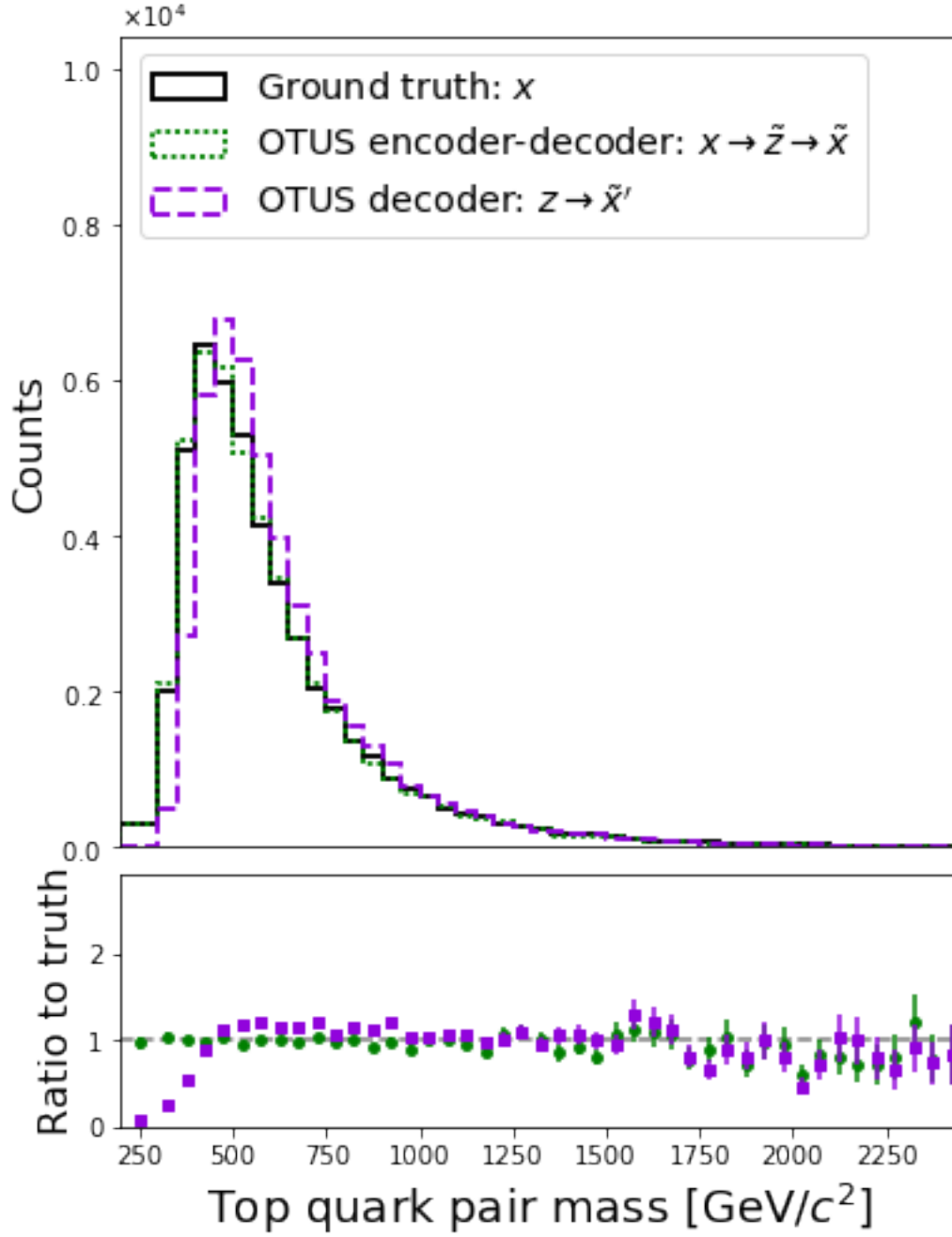
9.2 Make Plots for Invariant Mass Derived Quantities

```
[44]: ratioPlotDouble(ttbarPlotData[0], ttbarPlotData[1], ttbarPlotData[2],
    ↪ bins_ttbar,
    (200,2450), (0,5000), (0.00,2.95),
```

```

        r'Top quark pair mass [GeV/$c^2$]', legend=True,
        save_figs=save_figs, img_dir=img_dir) #xIMstatsList[0],
        xIMstatsListTruth[0],

```



```

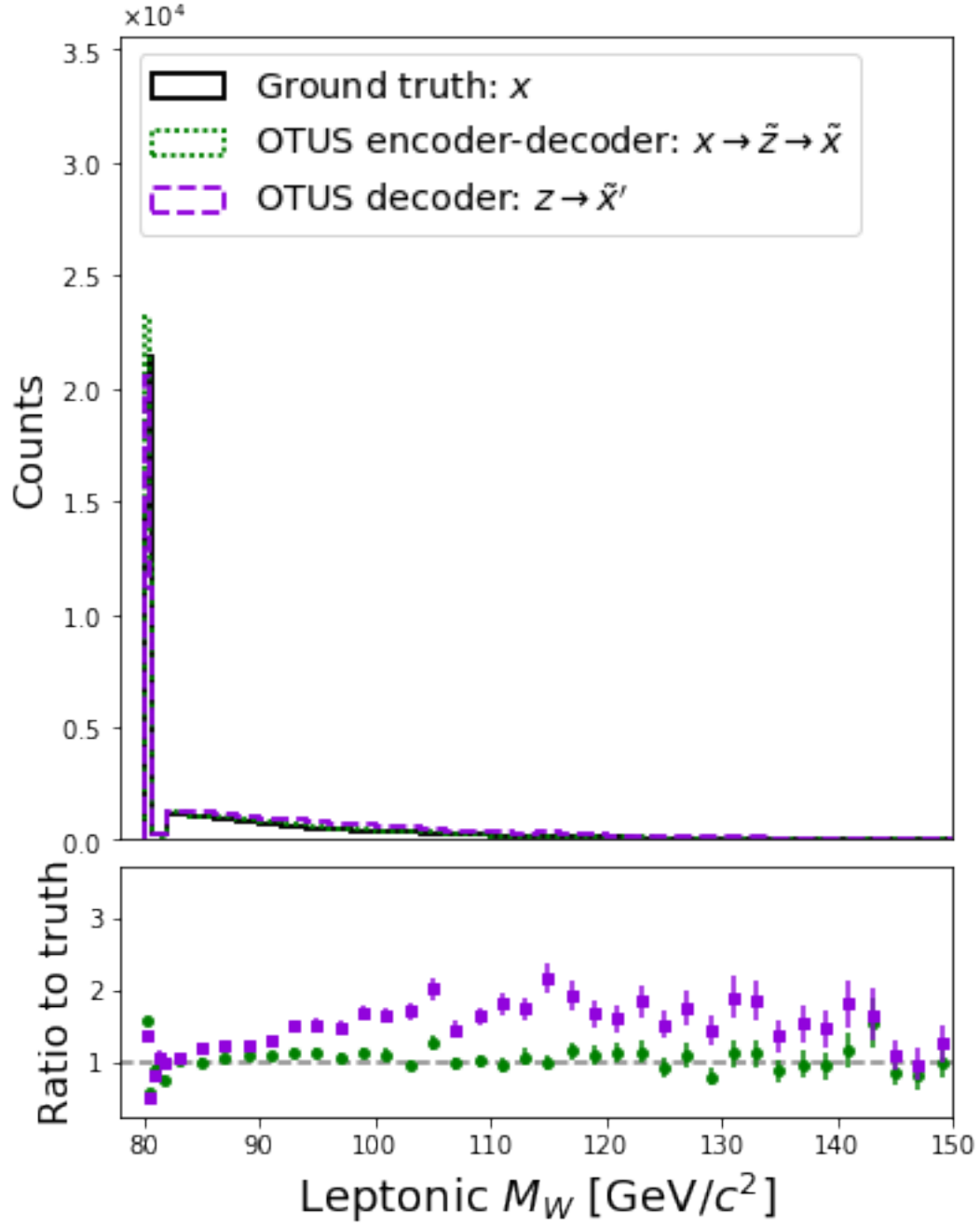
[45]: ratioPlotDouble(WlepPlotData[0], WlepPlotData[1], WlepPlotData[2], bins_Wlep,
                    (78,150), (0,21000), (0.25,3.75),

```

```

r'Leptonic  $M_W$  [GeV/ $c^2$ ]', legend=True,
save_figs=save_figs, img_dir=img_dir) #xIMstatsList[1],
xIMstatsListTruth[1],

```



```

[46]: ratioPlotDouble(WhadPlotData[0], WhadPlotData[1], WhadPlotData[2], bins_Whad,
(20,200), (0,10000), (0.25,4.95),

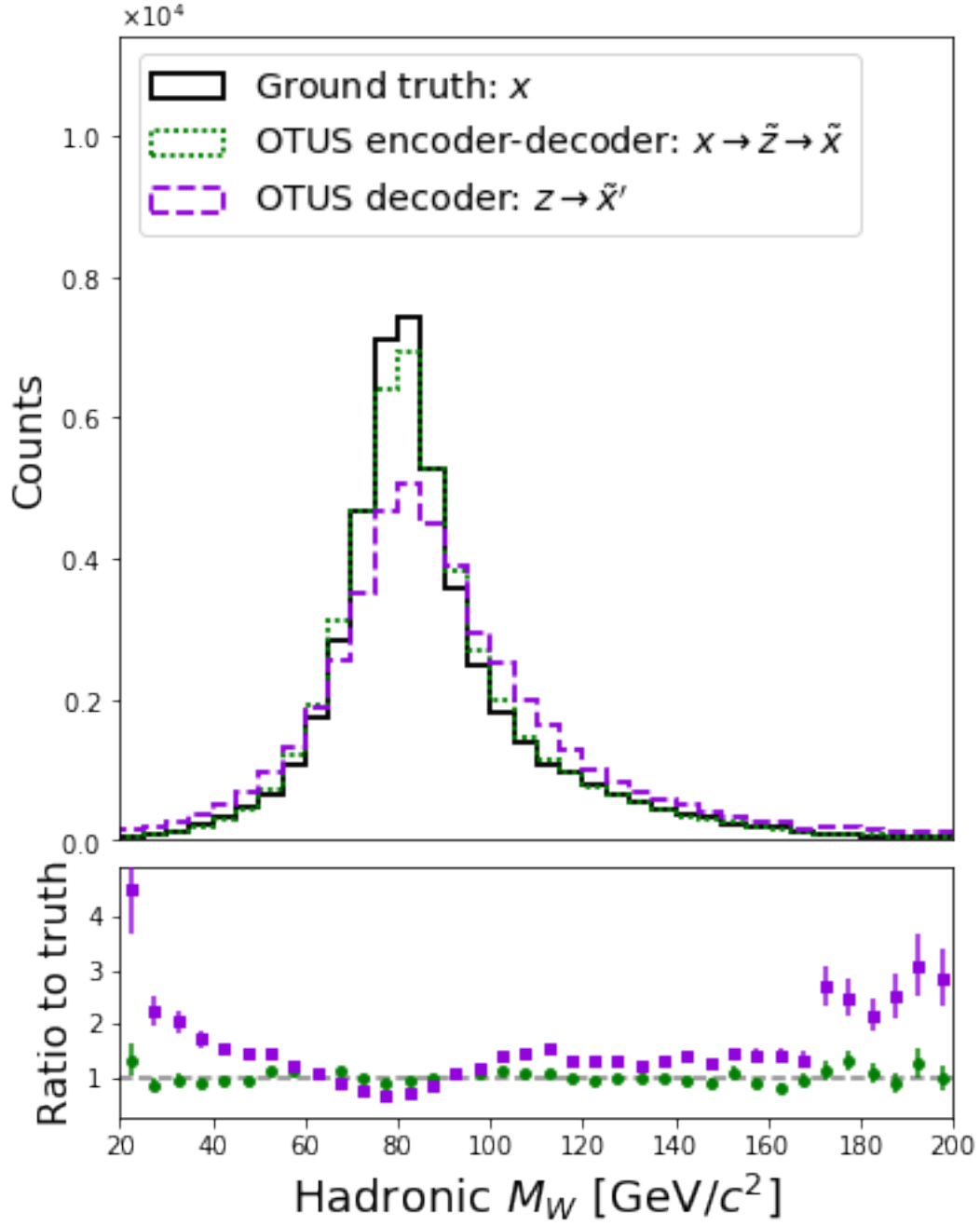
```



```

        r'Hadronic  $M_W$  [GeV/ $c^2$ ]', legend=True,
        save_figs=save_figs, img_dir=img_dir) #xIMstatsList[2],
        xIMstatsListTruth[2],

```



```

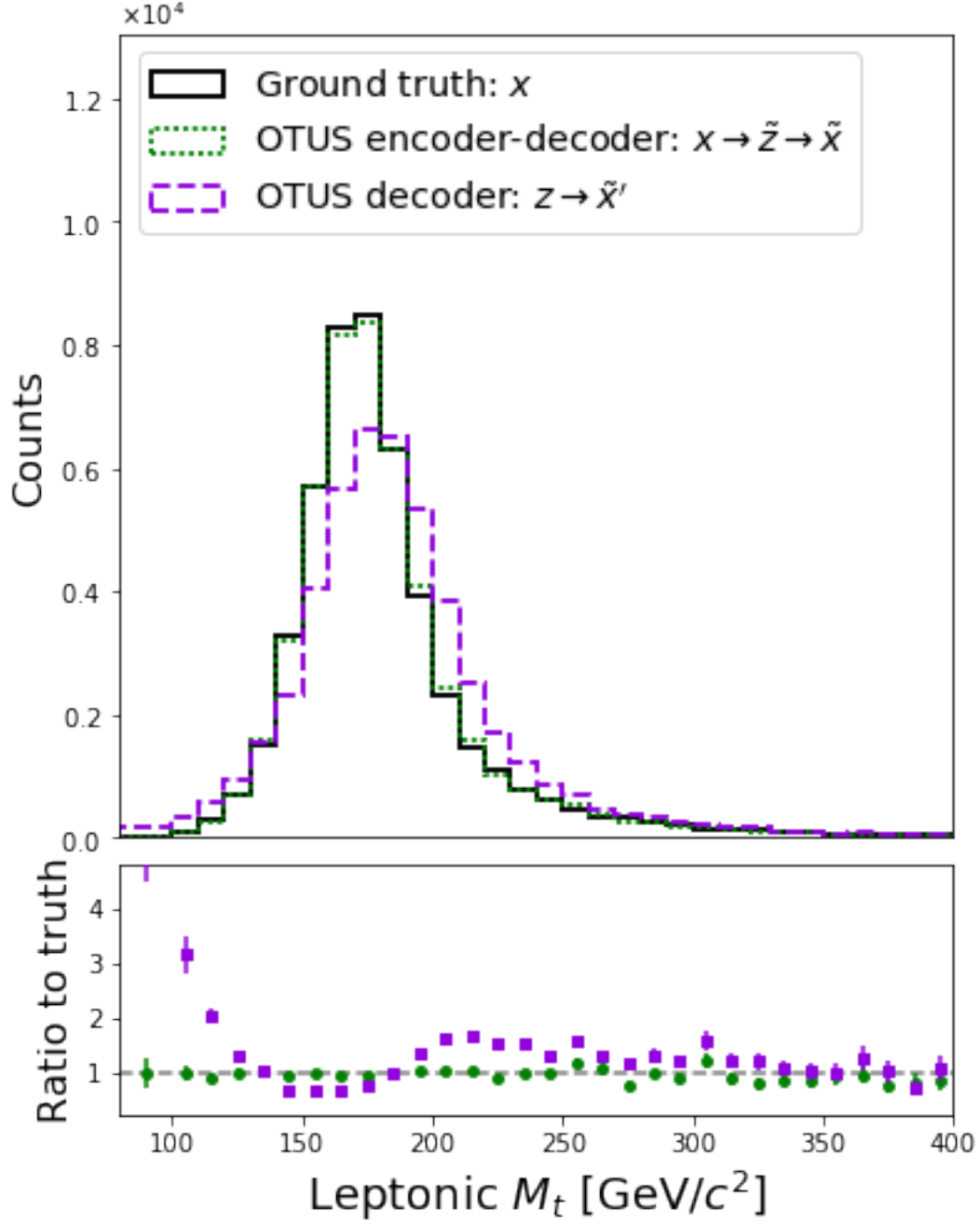
[47]: ratioPlotDouble(tlepPlotData[0], tlepPlotData[1], tlepPlotData[2], bins_tlep,
        (80,400), (0,10000), (0.25,4.85),

```

```

r'Leptonic  $M_t$  [GeV/ $c^2$ ]', legend=True,
save_figs=save_figs, img_dir=img_dir) #xIMstatsList[3],
xIMstatsListTruth[3],

```



```

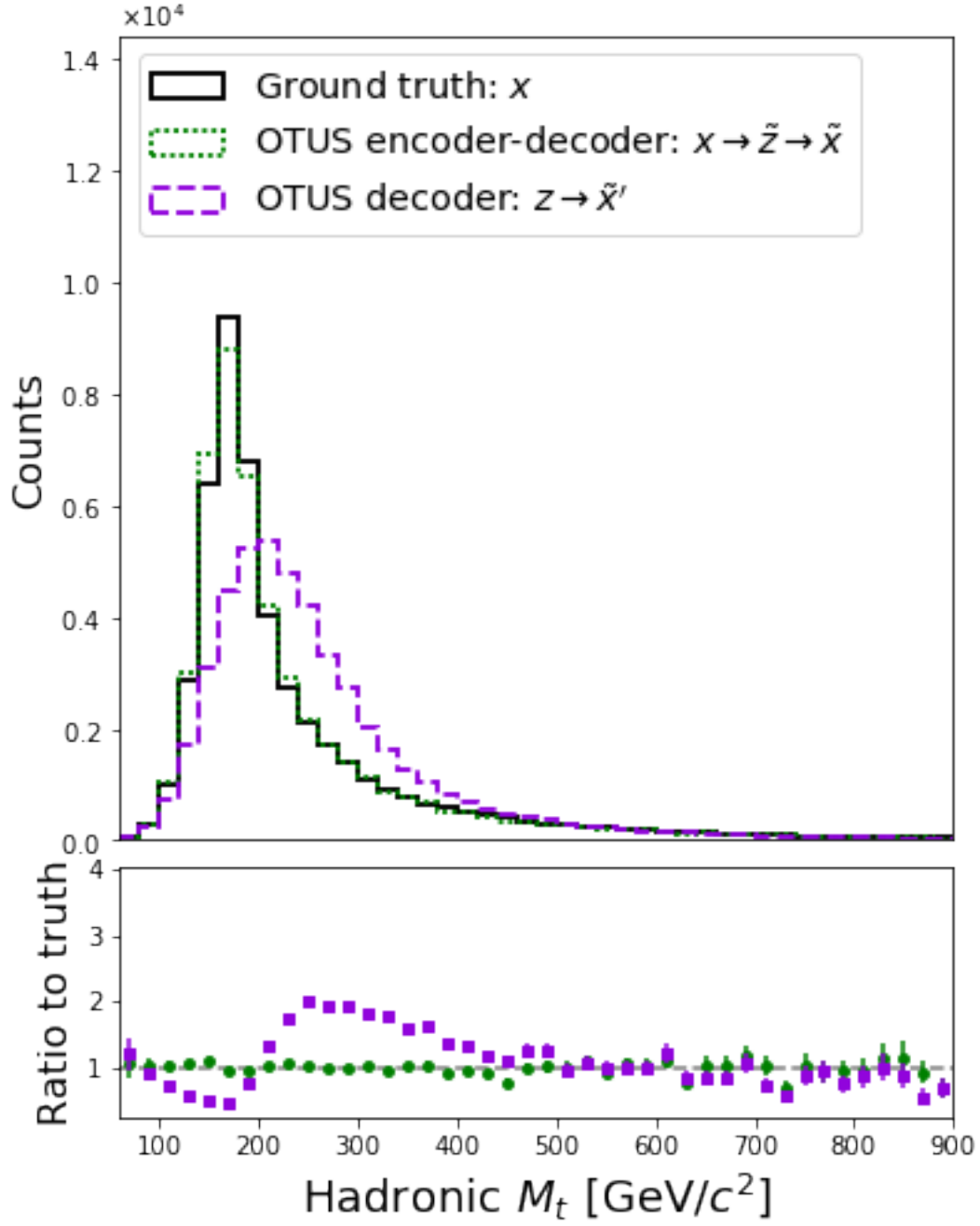
[48]: ratioPlotDouble(thadPlotData[0], thadPlotData[1], thadPlotData[2], bins_thad,
(60,900), (0,10000), (0.25,4.05),

```

```

r'Hadronic $M_{t}$ [GeV/$c^2$]', legend=True,
save_figs=save_figs, img_dir=img_dir) #xIMstatsList[4],
xIMstatsListTruth[4],

```



10 Transport plots

Brief Reminder of Notation:

- `TransportPlotData[0]` -> $[\tilde{z}, \tilde{x}]$
- `TransportPlotData[1]` -> $[z, x]$
- `TransportPlotData[2]` -> $[z, \tilde{x}']$

Ex. - $\tilde{z} = \text{TransportPlotData}[0][0] - \tilde{z}$, b quark energy = `TransportPlotData[0][0][:,11]` - $\tilde{x} = \text{TransportPlotData}[0][0] - \tilde{x}$, leading jet energy = `TransportPlotData[0][1][:,11]`

10.1 Decoder $\tilde{z} \rightarrow \tilde{x}$

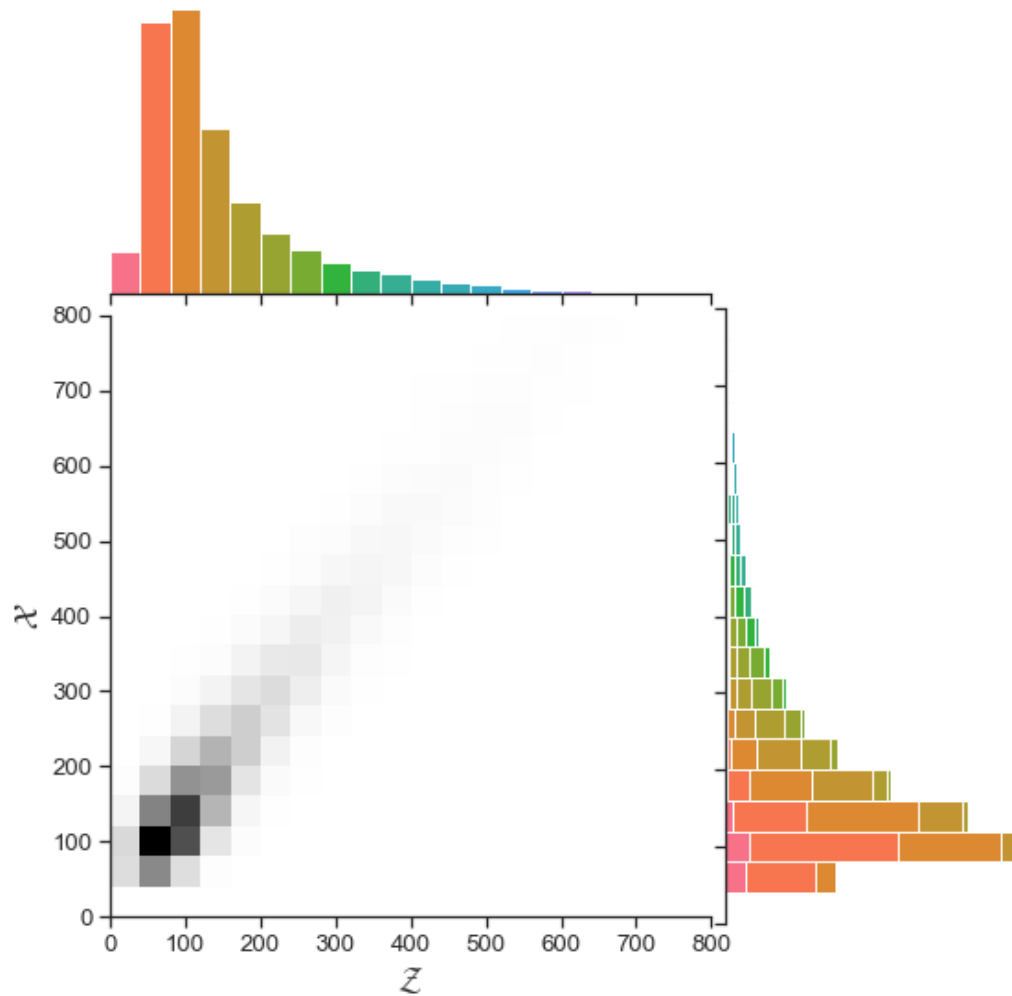
```
[49]: ## Make decoder transport plot from learned latent space##

# Leading jet energy
Z = TransportPlotData[0][0][:,11].reshape(-1,1)
X = TransportPlotData[0][1][:,11].reshape(-1,1)

n = Z.shape[1]
ZX = [np.concatenate((Z[:,i].reshape(-1,1), X[:,i].reshape(-1,1)), axis=1) for
      ↪ i in range(n)]

transportPlot(ZX[0], zlim=(0, 800), xlim=(0,800), nz=20, nx=20,
      ↪ name='decTransport_zpred_xpred_te.png', dec=True, title=r'$\hspace{4}$ OTUS
      ↪ encoder-decoder: $\tilde{z} \rightarrow \tilde{x}$', save_figs=save_figs,
      ↪ img_dir=img_dir)
```

OTUS encoder-decoder: $\tilde{Z} \rightarrow \tilde{X}$



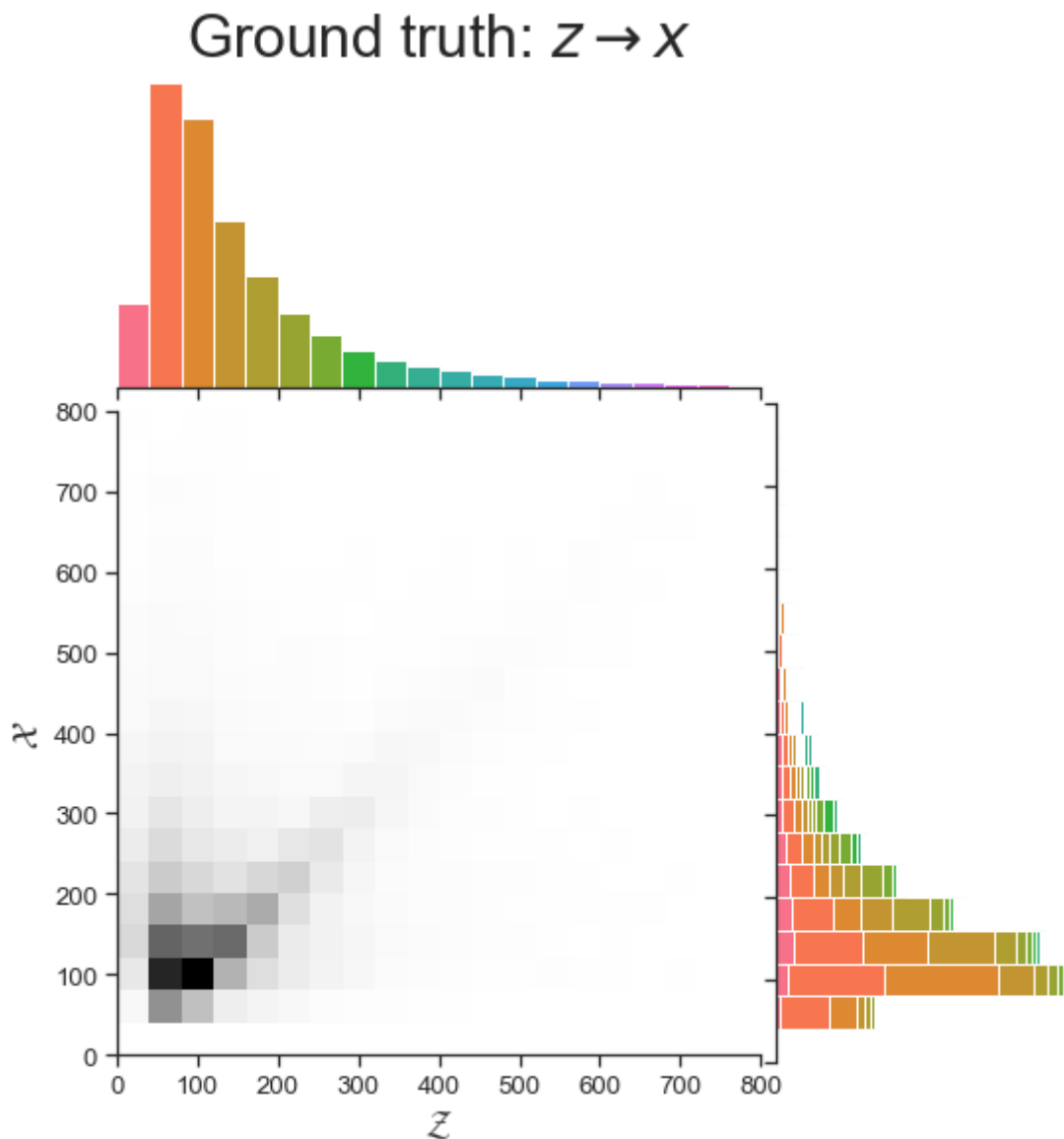
10.2 Delphes $z \rightarrow x$

```
[50]: ## Make delphes transport plot ##

# Leading jet energy
Z = TransportPlotData[1][0][:,11].reshape(-1,1)
X = TransportPlotData[1][1][:,11].reshape(-1,1)

n = Z.shape[1]
ZX = [np.concatenate((Z[:,i].reshape(-1,1), X[:,i].reshape(-1,1)), axis=1) for
      ↪ i in range(n)]
```

```
transportPlot(ZX[0], zlim=(0, 800), xlim=(0,800), nz=20, nx=20,
↳name='delphesTransport_ztest_xtest.png', dec=True, title=r'Ground truth: $z_{\perp}$
↳\rightarrow x$', save_figs=save_figs, img_dir=img_dir)
```



10.3 Decoder $z \rightarrow \tilde{x}'$

```
[51]: ##-- Make decoder transport plot from truth --#

# Leading jet energy
Z = TransportPlotData[2][0][:,11].reshape(-1,1)
X = TransportPlotData[2][1][:,11].reshape(-1,1)
```

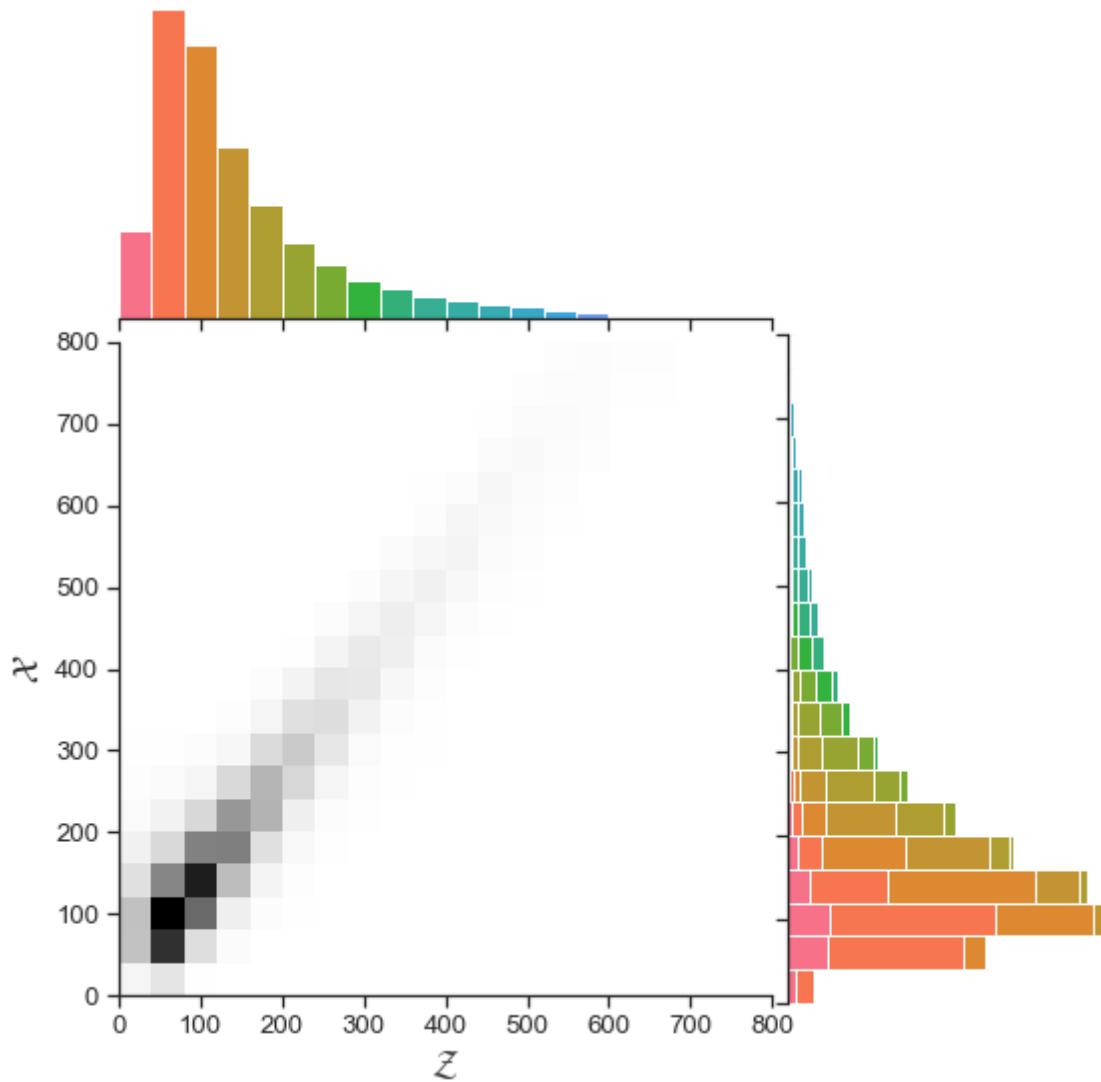
```

n = Z.shape[1]
ZX = [np.concatenate((Z[:,i].reshape(-1,1), X[:,i].reshape(-1,1)), axis=1) for
    i in range(n)]

transportPlot(ZX[0], zlim=(0,800), xlim=(0,800), nz=20, nx=20,
    name='decTransport_ztruth_xpredtruth_te.png', dec=True, title=r'OTUS decoder:
    $z \rightarrow \tilde{x}^\prime$, save_figs=save_figs, img_dir=img_dir)

```

OTUS decoder: $z \rightarrow \tilde{x}'$



11 Statistics Tables

We document the results of the above statistical tests in LaTeX table friendly format.

```
[52]: from func_utils import statTableSingle, statTableDouble
```

11.1 Supplementary Table 3 (z-space)

```
[53]: # Fig. 5a: fracDiff_zpred_z_bQuarkp_y.png
statTableSingle(statList12=zStatsList[0], figLabel='5a', cline=True)

# Fig. 5b: fracDiff_zpred_z_bQuarkp_z.png
statTableSingle(statList12=zStatsList[1], figLabel='5b', cline=True)

# Fig. 5c: fracDiff_zpred_z_bQuarkE.png
statTableSingle(statList12=zStatsList[2], figLabel='5c', cline=False)

\footnotesize{\textbf{Fig. 5a}} & \footnotesize{$1.58 \times 10^{+01}$} &
\footnotesize{($7.418$, $49$)} & \footnotesize{$1.25 \times
10^{-02}$} \\ \cline{2-4}
\footnotesize{\textbf{Fig. 5b}} & \footnotesize{$5.52 \times 10^{+01}$} &
\footnotesize{($4.613$, $55$)} & \footnotesize{$1.65 \times
10^{-02}$} \\ \cline{2-4}
\footnotesize{\textbf{Fig. 5c}} & \footnotesize{$6.20 \times 10^{+01}$} &
\footnotesize{($31.228$, $31$)} & \footnotesize{$4.04 \times 10^{-02}$} \\ \hline
```

11.2 Supplementary Table 4 (x-space)

```
[54]: # Fig. 5d: fracDiff_xpred_xpredtruth_x_LeadingJetp_y.png
statTableDouble(statList12=xStatsList[0], statList13=xStatsListTruth[0],
    figLabel='5d', cline=True)

# Fig. 5e: fracDiff_xpred_xpredtruth_x_LeadingJetp_z.png
statTableDouble(statList12=xStatsList[1], statList13=xStatsListTruth[1],
    figLabel='5e', cline=True)

# Fig. 5f: fracDiff_xpred_xpredtruth_x_LeadingJetE.png
statTableDouble(statList12=xStatsList[2], statList13=xStatsListTruth[2],
    figLabel='5f', cline=False)

# Fig. 7a: fracDiff_xpred_xpredtruth_x_Topquarkpairmass.png
statTableDouble(statList12=xIMstatsList[0], statList13=xIMstatsListTruth[0],
    figLabel='7a', cline=True)

# Fig. 7b: fracDiff_xpred_xpredtruth_x_HadronicM_W.png
statTableDouble(statList12=xIMstatsList[2], statList13=xIMstatsListTruth[2],
    figLabel='7b', cline=True)
```



```

# Fig. 7c: fracDiff_xpred_xpredtruth_x_LeptonicM_t.png
statTableDouble(statList12=xIMstatsList[3], statList13=xIMstatsListTruth[3],
  figLabel='7c', cline=True)

# Fig. 7d: fracDiff_xpred_xpredtruth_x_HadronicM_t.png
statTableDouble(statList12=xIMstatsList[4], statList13=xIMstatsListTruth[4],
  figLabel='7d', cline=False)

```

```

\footnotesize{\textbf{Fig. 5d}} & \footnotesize{\$2.40 \times 10^{+01}\$} &
\footnotesize{(\$2.395$, \$49\$)} & \footnotesize{\$1.66 \times 10^{-02}\$} &
\footnotesize{\$1.23 \times 10^{+02}\$} & \footnotesize{(\$34.021$, \$49\$)} &
\footnotesize{\$4.59 \times 10^{-02}\$}\\\cline{2-7}
\footnotesize{\textbf{Fig. 5e}} & \footnotesize{\$1.08 \times 10^{+02}\$} &
\footnotesize{(\$0.828$, \$55\$)} & \footnotesize{\$9.90 \times 10^{-03}\$} &
\footnotesize{\$3.42 \times 10^{+02}\$} & \footnotesize{(\$1.980$, \$55\$)} &
\footnotesize{\$6.94 \times 10^{-03}\$}\\\cline{2-7}
\footnotesize{\textbf{Fig. 5f}} & \footnotesize{\$4.11 \times 10^{+01}\$} &
\footnotesize{(\$1.281$, \$30\$)} & \footnotesize{\$1.02 \times 10^{-02}\$} &
\footnotesize{\$3.24 \times 10^{+02}\$} & \footnotesize{(\$50.072$, \$30\$)} &
\footnotesize{\$4.80 \times 10^{-02}\$}\\\hline
\footnotesize{\textbf{Fig. 7a}} & \footnotesize{\$1.60 \times 10^{+02}\$} &
\footnotesize{(\$1.192$, \$43\$)} & \footnotesize{\$7.63 \times 10^{-03}\$} &
\footnotesize{\$1.03 \times 10^{+03}\$} & \footnotesize{(\$54.598$, \$43\$)} &
\footnotesize{\$1.03 \times 10^{-01}\$}\\\cline{2-7}
\footnotesize{\textbf{Fig. 7b}} & \footnotesize{\$9.30 \times 10^{-01}\$} &
\footnotesize{(\$3.974$, \$35\$)} & \footnotesize{\$1.66 \times 10^{-02}\$} &
\footnotesize{\$1.04 \times 10^{+02}\$} & \footnotesize{(\$68.392$, \$35\$)} &
\footnotesize{\$1.09 \times 10^{-01}\$}\\\cline{2-7}
\footnotesize{\textbf{Fig. 7c}} & \footnotesize{\$8.83 \times 10^{+00}\$} &
\footnotesize{(\$1.579$, \$30\$)} & \footnotesize{\$5.91 \times 10^{-03}\$} &
\footnotesize{\$7.41 \times 10^{+01}\$} & \footnotesize{(\$92.533$, \$30\$)} &
\footnotesize{\$1.31 \times 10^{-01}\$}\\\cline{2-7}
\footnotesize{\textbf{Fig. 7d}} & \footnotesize{\$2.21 \times 10^{+01}\$} &
\footnotesize{(\$2.455$, \$41\$)} & \footnotesize{\$1.72 \times 10^{-02}\$} &
\footnotesize{\$1.11 \times 10^{+03}\$} & \footnotesize{(\$160.712$, \$41\$)} &
\footnotesize{\$2.35 \times 10^{-01}\$}\\\hline

```